

HPML

Deep-Dive Study Notes

Lectures 1 · 2 · 3 · 4 · 5 · 6 · 9 — Theory + Worked Numericals

Course	COMS E6998 — High-Performance Machine Learning
University	Columbia University · Spring 2026
Instructor	Dr. Kaoutar El Maghraoui
Exam	Monday May 11 · 7–9 PM · Schermerhorn 614
Format	Closed book · 1 cheat sheet · ~25–35 questions
Generated	May 10, 2026

What This Document Covers

This document provides deep theoretical explanations and fully worked numerical examples for the seven lectures that form the core of the HPML final exam. Every important concept is explained from first principles, with intuition, worked examples, and common pitfalls clearly called out.

Section	Topic	Key Numericals
L1	HPC Intro & AI Landscape	Compute scaling, energy costs, performance gains
L2	Performance Methodology	Amdahl's Law (5 worked examples), Roofline Model, Optimizer formulas
L3	Python & PyTorch Performance	Bytecode tracing, torch.compile stack, autograd math
L4	CUDA Basics	Thread indexing (1D/2D worked), memory hierarchy numbers
L5	Advanced CUDA	Coalescing, occupancy, tiled matmul, CUDA streams
L6	Distributed Deep Learning	Ring AllReduce BW, ZeRO memory (exact), pipeline bubbles
L9	FlashAttention & Efficient Transformers	IO tiling analysis, online softmax math, block size calc

LECTURE 1 — HPC Introduction & AI Landscape

1.1 Why HPML? The Compute Explosion

Training frontier AI models now requires computational resources that were unimaginable a decade ago. The table below shows the staggering growth from AlexNet (2012) to GPT-4 (2023):

Model	Year	Parameters	Compute (PF-days)	Est. Cost
AlexNet	2012	60 M	0.01	~\$1K
GPT-2	2019	1.5 B	40	~\$50K
GPT-3	2020	175 B	3,640	\$4–12M
GPT-4	2023	~1.8 T	~100,000	\$100M+

✓ Training costs have grown 10,000× in a decade — efficiency is now an economic necessity.

1.2 Training Energy Costs

Energy consumption has become a critical constraint. Note the super-linear scaling: a 3-order-of-magnitude increase in parameters causes a 4-order-of-magnitude increase in energy.

Model	Energy (MWh)	Equivalent Homes Powered
GPT-2	1.7	~170 homes for 1 day
Llama 2	688	~60K homes for 1 day
GPT-3	1,287	~112K homes for 1 day
PaLM	3,436	~300K homes for 1 day
GPT-4	51,773	~4.5M homes for 1 day

1.3 The Parallelism Shift — Where Performance Gains Come From

Single-core clock speeds plateaued around 2004 (the 'power wall'). Since then, ALL meaningful performance gains have come from parallelism, not transistor speed. This is why GPU computing and distributed systems are the foundation of modern ML:

Era	From Transistors	From Parallelism	Takeaway
~1997 (Tera scale)	32×	32×	Balanced
~2008 (Peta scale)	8×	128×	Parallelism dominates
~2020 (Exa scale)	1.5×	670×	Almost ALL from parallelism

✓ 670× gain from parallelism vs only 1.5× from transistors — this IS why you study HPML.

1.4 Inference Dominates Lifecycle Compute

A common misconception is that training is the expensive part. In reality:

- **Inference costs up to 90% of total lifecycle compute** — training only ~10%.
- For every ChatGPT request, the model must load ~140 GB of weights (Llama-3-70B) just to generate one token.
- KV cache size: 64 KB/token (8B model), 160 KB/token (70B model). At batch=8, 128K context → **64 GB of KV cache**.

1.5 GenAI Inference — Two Phases

Unlike traditional ML (stateless: input → output), GenAI inference is stateful:

Phase	Description	Bottleneck	Arithmetic Intensity
Prefill	All input tokens processed; KV cache populated	Compute-bound (large parallel matmuls)	High
Decode	One token generated at a time; reads full KV cache	Memory-bandwidth-bound	~20 FLOP/byte (vs H100 ridge ~625)

★ EXAM: Decode = memory-bandwidth-bound. Hardware utilization is ~3% of peak — the central engineering challenge.

1.6 HPC Design Principles for AI

Four core HPC principles, reimagined for AI superclusters:

- **Partition Model:** Separate service, compute, and I/O nodes. GPU nodes do no OS work.
- **Network Topology:** Torus or Dragonfly for HPC; NVLink/NVSwitch (900 GB/s) + InfiniBand (400 Gb/s) for AI clusters.
- **Balance of Components:** Compute speed must match memory bandwidth. Imbalance = GPU starvation.
- **Scalable System Software:** Minimal OS overhead on compute nodes; Parallel File System (GPFS) for I/O.

LECTURE 2 — Performance Methodology, Roofline & Optimizers

2.1 The 3-Step Optimization Cycle

Performance optimization is never a one-shot exercise. It is a **cyclical, iterative process**:

Step 1: MEASURE	Step 2: ANALYZE	Step 3: OPTIMIZE
• Execute workload • Profile with sampling or counters • Trace execution • Time sections precisely	• Identify Critical Path (highest Amdahl p) • Identify Bottleneck: compute vs memory • Apply Roofline Model • Understand data movement	• Cache blocking if memory-bound • Vectorization/SIMD if compute-bound • Reduce allocations • Fuse kernels → then go back to Step 1

2.2 Amdahl's Law — Complete Theory & 5 Worked Examples

Statement: The overall speedup of a system is fundamentally limited by the fraction of time that cannot be parallelized. No matter how fast you make the parallelizable portion, the serial fraction sets a hard ceiling.

$$S = 1 / [(1 - p) + p/s]$$

Variables: S = overall speedup of the application; p = fraction of execution time spent in the section being improved ($0 \leq p \leq 1$); s = speedup of that improved section; (1-p) = serial (unimproved) fraction.

Limiting case: As $s \rightarrow \infty$ (infinitely fast improved section), $S \rightarrow 1/(1-p)$. This is the absolute maximum speedup possible regardless of how good your optimization is.

Worked Example 1 — Basic Application with Two Sections

Application: Section A takes 75% of time, Section B takes 25%. What is the speedup if we make B 5× faster? What if we make A 2× faster?

```
Given: p_A = 0.75, p_B = 0.25, total time = 100s
Case 1: Make B 5× faster (s=5, p=0.25):
S = 1 / [(1 - 0.25) + 0.25/5]
S = 1 / [0.75 + 0.05]
S = 1 / 0.80 = 1.25× ← only 25% faster despite 5× optimization!
Case 2: Make A 2× faster (s=2, p=0.75):
S = 1 / [(1 - 0.75) + 0.75/2]
S = 1 / [0.25 + 0.375]
S = 1 / 0.625 = 1.60× ← 60% faster with only 2× optimization!
LESSON: Optimizing A (larger section) 2× beats optimizing B 5×.
ALWAYS profile first and optimize the CRITICAL PATH (high p).
```

Worked Example 2 — Parallel GPU Training (the Classic Case)

A neural network training loop: 90% of time is GPU matrix operations (parallelizable), 10% is data loading and CPU overhead (serial). How fast can we train with more GPUs?

```
Given: p = 0.90 (GPU compute), 1-p = 0.10 (serial overhead)
With 1 GPU (s=1): S = 1/[0.10 + 0.90/1] = 1/1.00 = 1.0×
With 4 GPUs (s=4): S = 1/[0.10 + 0.90/4] = 1/[0.10+0.225] = 1/0.325 = 3.08×
```

With 8 GPUs ($s=8$): $S = 1/[0.10 + 0.90/8] = 1/[0.10+0.1125] = 1/0.2125 = 4.71\times$
 With 16 GPUs ($s=16$): $S = 1/[0.10 + 0.90/16] = 1/[0.10+0.0563] = 1/0.1563 = 6.40\times$
 With 100 GPUs($s=100$): $S = 1/[0.10 + 0.90/100]= 1/[0.10+0.009] = 1/0.109 = 9.17\times$
 Theoretical max ($s\rightarrow\infty$): $S = 1/(1-0.90) = 1/0.10 = 10\times \leftarrow$ HARD CEILING
 Efficiency at 8 GPUs = $S/(\text{num_GPUs}) = 4.71/8 = 58.9\%$ [strong scaling efficiency]

✓ With 10% serial overhead, you can NEVER exceed 10× speedup, even with infinite GPUs!

Worked Example 3 — DDP Training with Communication Overhead

Synchronous DDP: 80% compute (forward+backward), 15% AllReduce gradient sync, 5% data loading. We can parallelize only the 80% compute portion.

Given: $p = 0.80$ (parallelizable compute), $1-p = 0.20$ (serial: comm + data load)
 With 1 GPU: $S = 1.0\times$
 With 4 GPUs: $S = 1/[0.20 + 0.80/4] = 1/[0.20+0.20] = 1/0.40 = 2.50\times$
 With 8 GPUs: $S = 1/[0.20 + 0.80/8] = 1/[0.20+0.10] = 1/0.30 = 3.33\times$
 With 16 GPUs: $S = 1/[0.20 + 0.80/16] = 1/[0.20+0.05] = 1/0.25 = 4.00\times$
 Max speedup ($s\rightarrow\infty$): $1/0.20 = 5.0\times$
 Efficiency at 8 GPUs = $3.33/8 = 41.6\% \leftarrow$ poor! Serial fraction too large.
 FIX: Overlap AllReduce with backward pass computation (PyTorch DDP does this)
 If overlap makes 'p' effectively 0.95: max speedup becomes $1/0.05 = 20\times$

Worked Example 4 — Maximum Speedup Calculation

What fraction of code must be parallelized to achieve a 10× speedup with 16 processors?

Given: $S_{\text{target}} = 10\times$, $s = 16$ processors
 Solve Amdahl's formula for p :
 $10 = 1 / [(1-p) + p/16]$
 $(1-p) + p/16 = 1/10 = 0.10$
 $1 - p + p/16 = 0.10$
 $1 - 0.90 = p - p/16 = p(1 - 1/16) = p(15/16)$
 $p = 0.90 \times (16/15) = 0.96$
 ANSWER: You need at least 96% of the code to be parallelizable to achieve 10× speedup with 16 processors.

Worked Example 5 — Scaling Efficiency Calculation

DDP training: serial time $T_{\blacksquare} = 400\text{s}$, parallel time $T_{\blacksquare} = 70\text{s}$ with 8 GPUs. What is the scaling efficiency? What is the implicit serial fraction?

Speedup: $S = T_{\blacksquare} / T_P = 400 / 70 = 5.71\times$
 Efficiency: $E = S / P = 5.71 / 8 = 71.4\%$
 Back-calculate serial fraction from Amdahl:
 $5.71 = 1 / [(1-p) + p/8]$
 $(1-p) + p/8 = 1/5.71 = 0.175$
 $1 - 7p/8 = 0.175$
 $7p/8 = 0.825$
 $p = 0.825 \times 8/7 = 0.943$
 Serial fraction = $1 - p = 5.7\% \rightarrow$ this is the AllReduce + overhead cost

To improve: reduce communication overhead with faster interconnect or better overlap of AllReduce with backward computation.

2.3 The Roofline Performance Model — Theory & Numericals

The Roofline model answers: **'Is my kernel limited by compute or memory bandwidth?'** It provides a visual framework showing the maximum achievable FLOPS as a function of Arithmetic Intensity.

Key Concepts

- **Arithmetic Intensity (AI)** = FLOP count / bytes transferred from DRAM [FLOP/byte]
- Small AI → memory-bound (limited by DRAM bandwidth)
- Large AI → compute-bound (limited by peak FLOPS)

$$\text{Attainable FLOPS} = \min(\text{Peak FLOPS}, \text{AI} \times \text{Memory Bandwidth})$$

$$\text{Crossover AI} = \text{Peak FLOPS} / \text{Memory Bandwidth}$$

Worked Example 1 — DAXPY (Double-Precision A·X + Y)

Code: $Z[i] = A * (X[i] + Y[i])$ for $i = 0..N-1$

Count FLOPs: 1 multiply ($A \times X[i]$) + 1 add ($+Y[i]$) = 2 FLOPs per iteration
Count bytes: Read $X[i]=8B$, Read $Y[i]=8B$, Write $Z[i]=8B$ = 24 bytes
(A is a scalar, loaded once and cached – not counted per iteration)
Arithmetic Intensity = 2 FLOP / 24 bytes = 0.083 FLOP/byte
Hardware (Columbia Xeon E5630 @ 2.53GHz):
Peak FLOPS = 2.53 GHz × 4 cores × 8 FP64/cycle = 81.3 GFLOPS
DRAM Bandwidth = 25.6 GB/s
Crossover AI = 81.3 / 25.6 = 3.17 FLOP/byte
Since AI = 0.083 < 3.17 → DAXPY is MEMORY-BOUND
Max attainable = 0.083 × 25.6 = 2.1 GFLOPS (out of 81.3 peak)
Utilization = 2.1/81.3 = 2.6% of peak compute – very poor!
FIX: Cache blocking can raise effective AI by reusing data from L1/L2.

Worked Example 2 — 7-Point 3D Stencil

Updates each 3D grid point using its 6 neighbors + itself.

Count FLOPs: 6 additions + 1 multiply-by-(-6.0) = 7 FLOPs per point
Count bytes: Read 7 unique double values (8B each) + write 1 = 8 reads × 8B = 64B
(new[] write = 8B, so total = 64+8 = 72B? Conservative: 64B from DRAM)
Arithmetic Intensity = 7 / 64 = 0.109 FLOP/byte
Crossover = 3.17 FLOP/byte (same hardware)
0.109 < 3.17 → MEMORY-BOUND (still!)
Max attainable = 0.109 × 25.6 = 2.8 GFLOPS
After cache blocking:
Only unique data fetched from DRAM; neighbors reused from L1
If L1 cache eliminates 75% of DRAM traffic: effective bytes = 16B
New AI = 7/16 = 0.44 FLOP/byte → still memory-bound but 4× closer to crossover

2.4 Mean Values — Which Mean to Use and When

Mean Type	Formula	Use When	ML Example
Arithmetic	$(1/n)\sum x_i$	Adding raw values (times, latencies)	Average epoch time: $(t_1 + t_2 + t_3)/3$
Harmonic	$n / \sum (1/x_i)$	Averaging rates/throughput	Average tokens/sec across runs
Geometric	$(\prod x_i)^{(1/n)}$	Averaging ratios and speedups	Average speedup across benchmarks

Numerical example: Three runs give speedups of 2x, 4x, 8x
 Arithmetic mean = $(2+4+8)/3 = 4.67\times \leftarrow$ WRONG for ratios
 Geometric mean = $(2 \times 4 \times 8)^{(1/3)} = 64^{(1/3)} = 4.0\times \leftarrow$ CORRECT

★ EXAM: Speedup/ratio $\rightarrow$ Geometric mean. Throughput/rates $\rightarrow$ Harmonic mean. Times $\rightarrow$ Arithmetic mean.

2.5 Strong vs. Weak Scaling — Theory & Examples

Scaling Type	Definition	Goal	DL Example	Efficiency Formula
Strong Scaling	Fixed total problem size; increase #GPUs	Reduce time for same problem	Train on same dataset faster with more GPUs	$E = T_1 / (P \times T_P)$
Weak Scaling	Problem size grows proportionally with #GPUs	Keep runtime constant as workload grows	Train larger model with more GPUs (each holds same per-GPU batch)	$E = T_1 / T_P$ (ideally = 1.0)

Strong scaling example: Fixed global batch = 1024 samples
 1 GPU: $T = 400s \rightarrow S = 1.0\times, E = 100\%$
 2 GPUs: $T = 210s \rightarrow S = 1.90\times, E = 95\%$
 4 GPUs: $T = 112s \rightarrow S = 3.57\times, E = 89\%$
 8 GPUs: $T = 65s \rightarrow S = 6.15\times, E = 76.9\%$
 16 GPUs: $T = 40s \rightarrow S = 10.0\times, E = 62.5\% \leftarrow$ efficiency drops with AllReduce overhead

2.6 Optimizer Theory — From SGD to Adam (with Numericals)

All modern optimizers for deep learning are built on the same foundation: **Exponential Moving Average (EMA)** of gradient statistics. Understanding EMA is the key to understanding all of Momentum, RMSProp, and Adam.

$$\text{EMA: } v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot g_t$$

With $\beta=0.9$, this effectively averages over the last $1/(1-0.9) = 10$ steps. With $\beta=0.99$, it averages over **100 steps**. Higher β = smoother, less reactive.

SGD (Baseline)

$$w_{t+1} = w_t - \alpha \cdot g_t \text{ where } g_t = \partial L / \partial w_t$$

Problem: noisy gradients cause oscillations. Each mini-batch gradient may point in slightly wrong direction. Learning rate must be small to avoid divergence $\rightarrow$ slow convergence.

Momentum

$$V_t = \beta \cdot V_{t-1} + (1-\beta) \cdot g_t \text{ [EMA of gradients]}$$

$$w_{t+1} = w_t - \alpha \cdot V_t$$

Intuition: Instead of taking a pure step in the current gradient direction, we accumulate a 'velocity' that remembers past directions. This damps oscillations in steep narrow valleys and accelerates descent along consistent directions.

RMSProp

$$S_t = \beta \cdot S_{t-1} + (1-\beta) \cdot g_t^2 \text{ [EMA of squared gradients]}$$

$$w_{t+1} = w_t - (\alpha / \sqrt{S_t + \epsilon}) \cdot g_t$$

Intuition: Divide the learning rate by the root-mean-square of recent gradients. Parameters that consistently receive large gradients get smaller effective LR (prevents explosion). Parameters with small gradients get larger effective LR (prevents vanishing). This is called 'per-parameter adaptive learning rates'.

Adam — Full Theory with Bias Correction

Adam = Momentum (1st moment) + RMSProp (2nd moment) + bias correction.

$$V_t = \beta \cdot V_{t-1} + (1-\beta) \cdot g_t \text{ [1st moment: mean of gradients]}$$

$$S_t = \beta \cdot S_{t-1} + (1-\beta) \cdot g_t^2 \text{ [2nd moment: mean of sq. gradients]}$$

$$\hat{V}_t = V_t / (1 - \beta^t) \text{ [bias-corrected 1st moment]}$$

$$\hat{S}_t = S_t / (1 - \beta^t) \text{ [bias-corrected 2nd moment]}$$

$$w_{t+1} = w_t - \alpha \cdot \hat{V}_t / (\sqrt{\hat{S}_t} + \epsilon)$$

Why bias correction? $\hat{V}_t = 0$ and $\hat{S}_t = 0$ (initialized to zero). At early steps (small t), V_t and S_t are biased toward zero. Dividing by $(1-\beta^t)$ compensates: at $t=1$ with $\beta=0.9$: correction factor = $1/(1-0.9) = 10 \rightarrow$ scales up $\hat{V}_t$ appropriately.

★ Worked Numerical Example: Adam Step-by-Step

Scenario: Single scalar parameter $w = 2.0$. Gradients at steps 1,2,3: $g = [0.5, 0.3, 0.8]$. Hyperparams: $\alpha=0.01$, $\beta=0.9$, $\beta=0.999$, $\epsilon=1e-8$. Starting: $\hat{V}=0$, $\hat{S}=0$.

```

=== STEP t=1 (g = 0.5) ===
V = 0.9 × 0 + (1-0.9) × 0.5 = 0 + 0.05 = 0.05
S = 0.999 × 0 + (1-0.999) × 0.25 = 0 + 0.00025 = 0.00025
Bias correction:
V_hat = 0.05 / (1 - 0.9^1) = 0.05 / 0.1 = 0.50
S_hat = 0.00025 / (1 - 0.999^1) = 0.00025 / 0.001 = 0.25
Update:
w = 2.0 - 0.01 × 0.50 / (sqrt(0.25) + 1e-8)
= 2.0 - 0.01 × 0.50 / 0.5
= 2.0 - 0.01 × 1.0 = 2.0 - 0.010 = 1.990

=== STEP t=2 (g = 0.3) ===
V = 0.9 × 0.05 + 0.1 × 0.3 = 0.045 + 0.03 = 0.075
S = 0.999 × 0.00025 + 0.001 × 0.09 = 0.0002498 + 0.00009 = 0.0003398
V_hat = 0.075 / (1 - 0.9^2) = 0.075 / 0.19 = 0.3947
S_hat = 0.0003398 / (1 - 0.999^2) = 0.0003398 / 0.001999 = 0.1700
Update:
w = 1.990 - 0.01 × 0.3947 / (sqrt(0.1700) + 1e-8)

```

```

= 1.990 - 0.01 × 0.3947 / 0.4123
= 1.990 - 0.01 × 0.957 = 1.990 - 0.00957 = 1.9804

=== STEP t=3 (g = 0.8) ===
V = 0.9 × 0.075 + 0.1 × 0.8 = 0.0675 + 0.08 = 0.1475
S = 0.999 × 0.0003398 + 0.001 × 0.64 = 0.0003395 + 0.00064 = 0.0009795
V = 0.1475 / (1 - 0.9³) = 0.1475 / 0.271 = 0.5443
= 0.0009795 / (1 - 0.999³) = 0.0009795 / 0.002997 = 0.3268
Update:
w = 1.9804 - 0.01 × 0.5443 / (√0.3268 + 1e-8)
= 1.9804 - 0.01 × 0.5443 / 0.5716
= 1.9804 - 0.01 × 0.9522 = 1.9804 - 0.00952 = 1.9709

Summary: w went 2.0 → 1.990 → 1.980 → 1.971 (moving toward minimum)
Note: Bias correction at t=1 was crucial – without it, V=0.05 would give
a much smaller step (0.05/0.5 = 0.1) vs correct (0.50/0.5 = 1.0)

```

✓ Bias correction is critical at early training steps. Adam without bias correction converges much more slowly at the start.

AdamW vs. Adam — The Critical Difference

Problem with Adam: L2 regularization $\lambda \|w\|^2$ added to loss → gradient includes λw term → weight decay gets *scaled* by the adaptive learning rate. For large-gradient parameters, weight decay becomes negligible. This is mathematically inconsistent with the intent of regularization.

Adam+L2: $w_{t+1} = w_t - (\alpha / (\sqrt{v_t} + \epsilon)) \cdot (g_t + \lambda \cdot w_t) \leftarrow$ decay scaled by $1/\sqrt{v_t}$

AdamW: $w_{t+1} = w_t - (\alpha / (\sqrt{v_t} + \epsilon)) \cdot g_t - \alpha \cdot \lambda \cdot w_t \leftarrow$ decay **INDEPENDENT**

✓ AdamW decouples weight decay from adaptive gradient scaling. De facto standard for all LLM training.

LECTURE 3 — Python & PyTorch Performance

3.1 Why Python is Slow — From First Principles

To understand why PyTorch needs a C++/CUDA backend, you must understand what Python actually does when you write $x = a + b$. This seemingly simple operation expands into many interpreter steps:

Step	What Python Actually Does	Cost
1	LOAD_FAST: push 'a' onto evaluation stack	~1–5 ns
2	LOAD_FAST: push 'b' onto evaluation stack	~1–5 ns
3	Type check: are a and b numbers? Call <code>__add__</code> ?	~10–20 ns
4	Reference count update on 'a' and 'b'	~5 ns
5	BINARY_ADD: dispatch to correct C function	~10 ns
6	Allocate new PyObject on heap (28 bytes per float!)	~50–100 ns
7	STORE_FAST: store result, update refcount	~5 ns
TOTAL	~7 operations for one addition	~80–150 ns

Compare: NumPy/BLAS 1000×1000 matmul = ~0.05s

Python nested loop matmul = ~45s

Speedup factor = ~900×

✓ **Solution: Push ALL computation into compiled C++/CUDA kernels. Python = glue code only.**

3.2 Memory Layout — Stack vs. Heap

Understanding where data lives explains many performance characteristics:

Memory Region	Speed	Size	Who Uses It	Python Impact
Registers	~1 cycle	64 bytes	CPU ALU intermediate results	Not accessible from Python
Stack	~3 cycles	~8 MB	Function frames, local vars in C	C/CUDA kernels use this
Heap	~100 cycles	GBs	Dynamic allocations (malloc)	ALL Python objects live here
DRAM	~100 cycles	GBs	Main memory	Large tensors, model weights

Every Python float is a **heap-allocated PyObject** (28 bytes). Contrast with C float (4 bytes on stack) — 7× more memory, much slower access due to pointer indirection.

3.3 Python Execution Pipeline — Source to Bytecode

Python compilation pipeline (happens on every import or source change):

Stage	Name	What Happens	Output
-------	------	--------------	--------

1	Lexing & Parsing	Tokenize source; build Concrete Syntax Tree (CST)	CST preserves ALL info including whitespace
2	AST Generation	Lower CST to Abstract Syntax Tree	AST removes syntactic sugar
3	Semantic Analysis	Resolve names to scopes (local/global/nonlocal/free)	Symbol table
4	CFG Construction	Lower AST to Control Flow Graph (basic blocks)	CFG
5	Bytecode Emission	Generate stack-machine instructions; package into PyCodeObject	Bytecode (.pyc)
6	Peephole Optimizer	Constant folding (2+2→4), dead code removal, short sequences	Optimized bytecode
7	PVM Execution	ceval.c executes bytecode instruction by instruction	Program output

Stack Machine Trace: $d = a + b * c$

Python's VM is a **stack-based machine**. Every operation pushes/pops from a stack. Let $a=2$, $b=3$, $c=4$. The compiled bytecode and stack state:

Instruction	Operand	Stack After	Stack State
LOAD_FAST	0 (a)	[a]	[2]
LOAD_FAST	1 (b)	[a, b]	[2, 3]
LOAD_FAST	2 (c)	[a, b, c]	[2, 3, 4]
BINARY_MULTIPLY	—	[a, b*c]	[2, 12]
BINARY_ADD	—	[a+b*c]	[14]
STORE_FAST	3 (d)	[]	d = 14, stack empty

3.4 torch.compile — The Modern Solution

PyTorch 2.0's **torch.compile** is a JIT compiler that captures computation graphs from Python bytecode and compiles them to optimized GPU kernels — with **zero code changes**:

```
model = torch.compile(model) # That's it!
```

Component	Role	Key Innovation	Output
TorchDynamo	Graph Acquisition	Hooks into CPython frame evaluation API at bytecode level. Captures FX graph without tracing. Uses GUARDS to validate specialization.	FX graph
AOTAutograd	Graph Lowering	Captures both forward AND backward graphs ahead-of-time. Enables joint optimization of training loop.	Joint fwd+bwd graph
TorchInductor	Graph Compilation	Generates Triton (GPU) or C++/OpenMP (CPU) kernels. Applies fusion, tiling, vectorization automatically.	Compiled kernels

- **Guards:** TorchDynamo inserts runtime checks (shape, dtype, device). If guard fails, it recompiles for the new specialization. This is why the first call is slow.
- **Kernel Fusion:** TorchInductor fuses multiple ops (matmul + relu + dropout) into one kernel, eliminating intermediate tensor allocations and repeated HBM loads.

✓ torch.compile speedups on A100: +38% TIMM, +76% TorchBench, +52% HuggingFace — zero code changes.

3.5 PyTorch Autograd — How Gradients are Computed

Autograd builds a dynamic computation graph during the forward pass, then traverses it backward using the chain rule:

$$\text{Chain rule: } d/dx[h(g(x))] = (dh/dg) \cdot (dg/dx)$$

Concrete example: $z = 2x^2 + 5x$, $x = [[2, 2], [2, 2]]$

Analytical gradient: $dz/dx = 4x + 5$

At $x = 2$: $dz/dx = 4(2) + 5 = 13$

PyTorch confirms: $x.grad = [[13, 13], [13, 13]]$

- **requires_grad=True:** tells autograd to track operations for gradient computation
- **loss.backward():** traverses graph backward via chain rule; accumulates gradients in .grad
- **torch.no_grad():** disables graph construction for weight updates (prevents tracking non-grad ops)

■ **MUST call .zero_grad() before each backward pass — gradients ACCUMULATE by default!**

LECTURE 4 — CUDA Basics

4.1 CPU vs. GPU — Why They're Different by Design

CPUs and GPUs are built for fundamentally different goals. Understanding this determines WHEN to use each and HOW to write efficient GPU code:

Property	CPU (Latency Optimized)	GPU (Throughput Optimized)
Silicon Area	~60% for control + cache, ~40% for compute	~90% for arithmetic units (CUDA cores)
Core Count	4–60 powerful cores	Thousands of simple cores
Cache	Large L1/L2/L3 (reduces memory latency)	Small, programmer-managed (shared memory)
Control	Branch prediction, out-of-order exec	Simple: no branch prediction, SIMT
Best for	Sequential, complex control flow	Massively parallel, data-regular tasks
Latency	~nanoseconds per op	~microseconds for kernel launch
Sequential code	~10× faster than GPU	~10× slower than CPU

4.2 CUDA Thread Hierarchy — Complete Theory

CUDA organizes threads in a three-level hierarchy. Each level maps to hardware:

Level	Maps to Hardware	Max Size	Key Properties
Thread	CUDA Core (SP)	1	Has own registers and local memory; smallest unit
Block	Streaming Multiprocessor (SM)	1024 threads	Threads share memory and can sync with <code>__syncthreads()</code>
Grid	GPU Device	$2^{31}-1 \times 65535 \times 65535$	One grid per kernel launch; blocks execute in any order
Warp	32 consecutive threads	32	Scheduling unit; all execute SAME instruction (SIMT)

4.3 Global Thread Index — Theory & Worked Examples

The most critical formula in CUDA. `threadIdx` is only unique within its block. To get a globally unique index, you must combine block and thread indices:

```
i = blockIdx.x * blockDim.x + threadIdx.x (1D)
Row = blockIdx.y*blockDim.y + threadIdx.y Col = blockIdx.x*blockDim.x + threadIdx.x
(2D)
```

Worked Example 1 — Basic 1D Index

```
Launch config: kernel<<<4, 256>>>(args) (4 blocks of 256 threads)
Total threads = 4 × 256 = 1024
Thread in Block 2, threadIdx.x = 3:
global_i = blockIdx.x * blockDim.x + threadIdx.x
= 2 * 256 + 3 = 515
```

```
Thread in Block 0, threadIdx.x = 0: global_i = 0
Thread in Block 0, threadIdx.x = 255: global_i = 255
Thread in Block 1, threadIdx.x = 0: global_i = 256
Thread in Block 3, threadIdx.x = 255: global_i = 1023
```

Worked Example 2 — kernel<<<3,4>>> Traces

For launch config kernel<<<3,4>>>(a) — 3 blocks, 4 threads/block, 12 total threads:

Kernel Code	Result Array (a[0]...a[11])	Explanation
a[i] = blockDim.x	4 4 4 4 4 4 4 4 4 4 4 4	blockDim.x=4 for every thread in every block
a[i] = threadIdx.x	0 1 2 3 0 1 2 3 0 1 2 3	threadIdx resets to 0 at each block boundary
a[i] = blockIdx.x	0 0 0 0 1 1 1 1 2 2 2 2	blockIdx same for all threads in a block
a[i] = i (global)	0 1 2 3 4 5 6 7 8 9 10 11	unique global index = blockIdx×blockDim+threadIdx

★ EXAM: COMMON BUG: using a[i] = threadIdx.x only gives 0,1,2,3,0,1,2,3... NOT unique global indices!

Worked Example 3 — 2D Image Indexing

```
Image: 8×8 pixels. Launch: <<>>
blockDim.x=4, blockDim.y=4, gridDim.x=2, gridDim.y=2
Thread at blockIdx=(1,0), threadIdx=(1,2):
Row = blockIdx.y * blockDim.y + threadIdx.y = 0*4 + 2 = 2
Col = blockIdx.x * blockDim.x + threadIdx.x = 1*4 + 1 = 5
Linear index (row-major) = Row * width + Col = 2*8 + 5 = 21
So this thread processes pixel Image[2][5] = Image[21]
```

4.4 CUDA Memory Model — Complete Hierarchy

Level	Location	Latency	BW	Scope	Managed By	A100 Size
Registers	On-chip (per SM)	~1 cycle	~8 TB/s	Per thread (private)	Compiler	~256 KB/SM
Shared Mem / L1	On-chip (per SM)	~20 cycles	~1.5 TB/s	Per block	Programmer	192 KB/SM
L2 Cache	On-chip (GPU die)	~200 cycles	—	All SMs	Hardware	40 MB
Global (HBM)	Off-chip	~400–600 cycles	2 TB/s	All SMs	Programmer	80 GB
Constant	Off-chip (cached)	~1 (cached)	—	All (read-only)	Programmer	64 KB

✓ Register spilling = variables that don't fit in registers go to 'local memory' = ACTUALLY global memory = SLOW.

✓ Shared memory is programmer-managed L1 cache. Use `__shared__` to declare. Must sync with `__syncthreads()`.

4.5 CUDA Programming Pattern

```
// 1. Allocate device memory
cudaMalloc(&d_data, N * sizeof(float));
// 2. Copy Host → Device
cudaMemcpy(d_data, h_data, N*sizeof(float), cudaMemcpyHostToDevice);
// 3. Launch kernel (asynchronous!)
dim3 grid(N/256+1), block(256);
my_kernel<<>>(d_data, N);
// 4. Synchronize (wait for GPU)
cudaDeviceSynchronize();
// 5. Copy Device → Host
cudaMemcpy(h_data, d_data, N*sizeof(float), cudaMemcpyDeviceToHost);
// 6. Free device memory
cudaFree(d_data);
```

■ Kernel launches are **ASYNCHRONOUS**. `cudaMemcpy()` is **SYNCHRONOUS**. ALWAYS `cudaDeviceSynchronize()` before accessing results.

LECTURE 5 — Advanced CUDA

5.1 GPU Architecture Numbers

Spec	V100 (Volta)	A100 (Ampere)	H100 (Hopper)
Year	2017	2020	2022
SMs	80	108	132
CUDA Cores/SM	64+64	128	128
Total CUDA Cores	5,120	6,912 (actually 108×128)	18,432
Tensor Core Gen	1st/2nd	3rd (TF32,BF16,FP64,2:4)	4th (+FP8, TMA)
Peak FP32	15.7 TFLOPS	19.5 TFLOPS	60 TFLOPS
Peak BF16 TC	—	312 TFLOPS	1,000 TFLOPS
HBM BW	900 GB/s (HBM2)	2 TB/s (HBM2e)	3.35 TB/s (HBM3)
L2 Cache	6 MB	40 MB	50 MB
Shared Mem/SM	96 KB	192 KB	256 KB
NVLink BW	300 GB/s	600 GB/s	900 GB/s

5.2 Tensor Cores — Theory & Speedup Numbers

Tensor Cores perform $\mathbf{D} = \mathbf{A} \times \mathbf{B} + \mathbf{C}$ (matrix multiply-accumulate) in one operation. A, B in low precision (FP16/BF16/INT8/FP8), C/D in higher precision (FP32). They operate on 16×16 matrices per cycle — far more throughput than general CUDA cores.

GPU	Tensor Core Gen	FP16 Tensor TFLOPS	Speedup vs. No Tensor Cores
V100	1st gen	125 TFLOPS	4×
A100	3rd gen	312 TFLOPS (FP16), 624 (INT8)	8×
H100	4th gen	1,000 TFLOPS (FP16), 2,000 (FP8)	16×

✓ H100 with FP8 delivers 2,000 TFLOPS — 128× more throughput than FP32 CUDA cores alone. This is why FP8 training matters.

5.3 Memory Coalescing — Theory & Worked Examples

When threads in a warp access memory, the GPU hardware tries to coalesce (combine) multiple accesses into a single memory transaction. Proper coalescing is one of the most important optimizations for global memory performance.

Rule: Consecutive threads should access consecutive addresses

Pattern	Access Pattern	Transactions	Throughput	Example
Stride-1 (optimal)	Thread i → address[i*1]	1 transaction for 32 threads	100% (32 elements/tx)	float A[32] where thread i reads A[i]

Stride-2	Thread $i \rightarrow \text{address}[i*2]$	2 transactions	50%	Reading every other element
Stride-4	Thread $i \rightarrow \text{address}[i*4]$	4 transactions	25%	Reading every 4th element
Stride-32 (worst)	Thread $i \rightarrow \text{address}[i*32]$	32 transactions	3.1% (1 element/tx)	Classic matrix column access

Worked Example — Matrix Access Patterns

Matrix $A[1024][1024]$, stored row-major. Warp of 32 threads.

Case 1: Row access (GOOD): Thread i reads $A[\text{row}][i]$

Addresses: $A[\text{row}][0], A[\text{row}][1], \dots, A[\text{row}][31]$

Memory offsets: 0, 4, 8, ..., 124 bytes $\rightarrow$ stride-1 floats

$\rightarrow$ 1 cache line (128 bytes) covers all 32 threads $\rightarrow$ 1 transaction

Case 2: Column access (BAD): Thread i reads $A[i][\text{col}]$

Addresses: $A[0][\text{col}], A[1][\text{col}], \dots, A[31][\text{col}]$

Memory offsets: 0, 4096, 8192, ..., 126976 bytes $\rightarrow$ stride-1024 floats

$\rightarrow$ Each thread accesses a different cache line $\rightarrow$ 32 transactions

$\rightarrow$ 32 $\times$ more memory bandwidth consumed

★ EXAM: Pattern A (stride-1) = 1 transaction. Pattern B (stride-32) = 32 transactions. 32 $\times$ difference in effective bandwidth.

5.4 Shared Memory Bank Conflicts — Theory

Shared memory is divided into 32 banks (4-byte granularity). Consecutive 4-byte words go to consecutive banks (bank = address/4 mod 32).

- **No conflict:** All 32 threads access different banks $\rightarrow$ single cycle (ideal)
- **Bank conflict:** 2+ threads access different addresses in the SAME bank $\rightarrow$ serialized
- **Broadcast:** Multiple threads access the SAME address in the SAME bank $\rightarrow$ no conflict (broadcast)

Example: `__shared__ float tile[32][32]`

Thread i accesses $\text{tile}[0][i]$: $i=0 \rightarrow \text{bank } 0, i=1 \rightarrow \text{bank } 1, \dots, i=31 \rightarrow \text{bank } 31 \rightarrow$ NO CONFLICT

Thread i accesses $\text{tile}[i][0]$: $i=0 \rightarrow \text{addr } 0 \rightarrow \text{bank } 0, i=1 \rightarrow \text{addr } 128 \rightarrow \text{bank } 0 \rightarrow$ 32-WAY CONFLICT!

FIX: `__shared__ float tile[32][33]` $\rightarrow$ padding shifts column 0 to different banks

5.5 SM Occupancy — Theory & Worked Numerical

Occupancy = (active warps/SM) / (max warps/SM). High occupancy helps hide memory latency by having other warps ready to execute while one waits.

$$\text{Active Warps per SM} = (\text{threads_per_block} / 32) \times \text{blocks_per_SM}$$

Worked Example — Fermi Architecture (max 1536 threads/SM, max 8 blocks/SM, max 48 warps/SM)

Block Config	Threads/Block	Warps/Block	Max Blocks Fit	Active Warps	Occupancy	Verdict
8 $\times$ 8	64	2	8	2 $\times$ 8=16	16/48=33%	POOR — wasted SM resources

16×16	256	8	6 (limited by 6×256=1536≤1536)	8×6=48	48/48=100%	OPTIMAL
32×32	1024	32	1 (1×1024=1024≤1536, but max 8 blocks)	32×1=32	32/48=67%	Good but not optimal
12×12	144	4.5→5	8	5×8=40→but only 5 complete warps	35/48=73%	OK

✓ 16×16 block = 256 threads = 8 warps → 6 blocks fit on Fermi SM = 100% occupancy. This is the design sweet spot.

5.6 CUDA Streams — Overlapping Compute and Transfer

A CUDA Stream is an ordered queue of GPU work. Operations in the same stream execute sequentially. Operations in **different streams can execute concurrently** — this is the key to overlapping H2D transfers with kernel execution:

```
cudaStream_t s1, s2;
cudaStreamCreate(&s1); cudaStreamCreate(&s2);
// Stream 1: copy chunk A then process it
cudaMemcpyAsync(d_a, h_a, size, H2D, s1);
kernel_A<<>>(d_a);
// Stream 2: concurrently copy chunk B (while Stream 1's kernel runs!)
cudaMemcpyAsync(d_b, h_b, size, H2D, s2);
kernel_B<<>>(d_b);
```

- Pinned memory (cudaMallocHost) is REQUIRED for cudaMemcpyAsync to work correctly
- Unpinned memory forces blocking transfer — no overlap possible

LECTURE 6 — Distributed Deep Learning

6.1 Why Distributed Training Is Necessary — The Numbers

Model	Parameters	Memory (FP16)	Needed	Adam (FP32)	Optimizer	Single H100?
LLaMA-3 7B	7B	14 GB		~112 GB		Barely fits with ZeRO
LLaMA-3 70B	70B	140 GB		~1.1 TB		IMPOSSIBLE without ZeRO
LLaMA-3 405B	405B	810 GB		~6.5 TB		Need 80+ H100s with ZeRO-3
GPT-3 175B	175B	350 GB		~2.8 TB		Need 35+ H100s with ZeRO-3
GPT-4 ~1.8T	~1.8T	~3.6 TB		~29 TB		Need 1000+ H100s

Adam optimizer memory breakdown per parameter (16 bytes total):

FP16 model weights: 2 bytes

FP16 gradients: 2 bytes

FP32 master weights: 4 bytes (kept for numerical stability)

FP32 momentum (V): 4 bytes

FP32 variance (S): 4 bytes

TOTAL: 16 bytes per parameter

For 175B params: $175B \times 16 = 2,800 \text{ GB} = 2.8 \text{ TB}$

6.2 Collective Communication Primitives — Theory

Primitive	Direction	What Happens	DL Application	Bandwidth
Broadcast	1→all	Root sends SAME data to all ranks	Distribute model params at init	$N \times (P-1)$
Reduce	all→1	All ranks send; root combines (sum/max)	Gradient accumulation at PS	$N \times (P-1)$
AllReduce	all→all	Reduce then distribute; ALL get result	Gradient sync in DDP — most critical	$2N(P-1)/P$
Reduce-Scatter	all→all (partial)	Each rank gets a different shard of result	ZeRO gradient sharding	$N(P-1)/P$
AllGather	all→all (full)	Each has partial; all receive full data	ZeRO parameter reconstruction	$N(P-1)/P$
AllToAll	all→all (different)	Each rank sends different data to each rank	MoE expert routing	$N \times P$

★ EXAM: AllReduce = most important collective. Used in DDP for gradient averaging. Answer: Ring AllReduce.

6.3 Ring AllReduce — Full Algorithm with Numerics

Ring AllReduce is bandwidth-optimal. All P GPUs arranged in a logical ring. Each GPU sends only to its right neighbor. Two phases — $P-1$ steps each:

Phase 1: Scatter-Reduce ($P-1$ steps)

Each GPU sends a chunk to its right neighbor and accumulates from its left neighbor. After $P-1$ steps, each GPU has ONE fully-reduced chunk (the sum across all GPUs for that chunk).

Phase 2: AllGather ($P-1$ steps)

GPUs circulate the fully-reduced chunks around the ring. After $P-1$ steps, every GPU has ALL fully-reduced chunks → complete result everywhere.

Worked Bandwidth Example

```
Setup: P=4 GPUs, N=1B parameters (4 GB total in FP32)
Each GPU's gradient array split into P=4 chunks of N/P = 1 GB each
Phase 1 (Scatter-Reduce): 3 steps
Step 1: Each GPU sends chunk[0] → neighbor (1 GB sent per GPU)
Step 2: Each GPU sends accumulated chunk → neighbor (1 GB sent per GPU)
Step 3: Each GPU sends accumulated chunk → neighbor (1 GB sent per GPU)
Total sent per GPU: 3 × 1 GB = 3 GB = N(P-1)/P = 4GB × 3/4 = 3 GB ✓
Phase 2 (AllGather): 3 more steps
Same amount: 3 GB sent per GPU
Total per GPU = 2 × N(P-1)/P = 2 × 3 GB = 6 GB
Formula: 2N(P-1)/P = 2 × 4GB × (4-1)/4 = 2 × 3 GB = 6 GB ✓
Compare with Parameter Server (P=4):
PS receives from 3 workers: 3 × 4 GB = 12 GB RX
PS sends to 3 workers: 3 × 4 GB = 12 GB TX
Total PS node: 24 GB = 2N(P-1) = 2×4×3 = 24 GB
Ring AllReduce: 6 GB per GPU (independent of P for large P → scales!)
Parameter Server: 24 GB at PS (4× more, grows with P → bottleneck!)
```

✓ Ring AllReduce communication per GPU = $2N(P-1)/P \approx 2N$ for large P — essentially CONSTANT regardless of cluster size.

6.4 ZeRO Optimizer — Memory Calculation with Exact Numbers

ZeRO (Zero Redundancy Optimizer) shards model states across GPUs while maintaining data-parallel semantics (forward/backward pass unchanged).

Setup for All Examples

```
Model:  $\psi$  = 7.5B parameters, Adam optimizer (K=12 bytes for opt states per param)
N = 64 GPUs, each with 80 GB HBM
Baseline (naive DP): (2+2+12) × 7.5B = 16 × 7.5B = 120 GB per GPU
→ CANNOT FIT on 80 GB GPU! Need ZeRO.
```

ZeRO Stage 1: Shard Optimizer States Only

```
Each GPU stores: full weights (2B) + full gradients (2B) + 1/N optimizer states (12/N B)
Memory formula: (2 + 2 + 12/64) × 7.5B = (2 + 2 + 0.1875) × 7.5B
```

$= 4.1875 \times 7.5B = 31.4 \text{ GB per GPU} \checkmark$ (fits in 80 GB)
 Memory reduction: $120 / 31.4 = \sim 3.8\times \approx 4\times$ reduction
 Max trainable model: $80 \text{ GB} / 4.1875 = \sim 19B$ parameters

ZeRO Stage 2: Shard Optimizer States + Gradients

Each GPU stores: full weights (2B) + 1/N gradients (2/N B) + 1/N optimizer states (12/N B)
 Memory formula: $(2 + 2/64 + 12/64) \times 7.5B = (2 + 0.03125 + 0.1875) \times 7.5B$
 $= 2.21875 \times 7.5B = 16.6 \text{ GB per GPU} \checkmark$
 Memory reduction: $120 / 16.6 = \sim 7.2\times \approx 8\times$ reduction
 Max trainable model: $80 \text{ GB} / 2.22 = \sim 36B$ parameters

ZeRO Stage 3 (FSDP): Shard Everything

Each GPU stores: 1/N weights + 1/N gradients + 1/N optimizer states
 Memory formula: $(2/64 + 2/64 + 12/64) \times 7.5B = (16/64) \times 7.5B = 0.25 \times 7.5B$
 $= 1.875 \text{ GB per GPU} \leftarrow$ incredible reduction!
 Memory reduction: $120 / 1.875 = 64\times = N\times$ reduction (perfect sharding)
 Max trainable model: $80 \text{ GB} / 0.25 = 320B$ parameters

ZeRO Stage	Sharded	Mem/GPU (N=64)	Reduction	Max Model Size
Baseline (DDP)	Nothing	120 GB	1×	5B
ZeRO-1	Optimizer states	31.4 GB	~4×	19B
ZeRO-2	Optim + Gradients	16.6 GB	~8×	36B
ZeRO-3 (FSDP)	Optim + Grad + Params	1.9 GB	64×	320B

6.5 Pipeline Parallelism — Bubble Fraction Formula

Pipeline parallelism splits model layers across GPUs. Microbatches fill the pipeline to keep GPUs busy. The key challenge is the 'bubble' — idle GPU time at pipeline start and end.

Bubble fraction = $(P-1) / M$ where P = pipeline stages, M = microbatches

Ideal speedup = P (never reached due to bubble)

Example: $P=4$ stages, $M=8$ microbatches
 Bubble fraction = $(4-1)/8 = 3/8 = 37.5\%$ idle time
 With $M=16$ microbatches: bubble = $3/16 = 18.75\%$
 With $M=32$ microbatches: bubble = $3/32 = 9.4\%$
 As $M \rightarrow \infty$: bubble $\rightarrow 0$ (but memory explodes for GPipe since $O(M)$ activations)
 1F1B variant: $O(P)$ memory instead of $O(M)$ — much better for long pipelines

LECTURE 9 — Efficient Transformers & FlashAttention

9.1 Self-Attention — Complete Theory

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Step-by-step intuition:

- **Q (Query):** 'What am I looking for?' — the current token asking a question
- **K (Key):** 'What information do I represent?' — every token advertising itself
- **V (Value):** 'What content do I provide if selected?' — the actual information to extract
- **Score_{ij} = Q_i · K_j^T / $\sqrt{d_k}$:** How relevant is token j to token i? Divide by $\sqrt{d_k}$ to prevent softmax saturation.
- **softmax:** Normalize scores to probability distribution (sum=1) over all tokens
- **× V:** Weighted sum of all values, weighted by how relevant each token is

```
Concrete example: d_k=64, so sqrt(d_k)=8
Q_1 · K_1 = 112 → divide by 8 → score=14
Q_1 · K_2 = 96 → divide by 8 → score=12
softmax([14, 12]) = [e^14, e^12] / (e^14 + e^12) = [0.88, 0.12]
Output_1 = 0.88 × V_1 + 0.12 × V_2 (mostly V_1)
```

Standard Complexity:

- Time complexity: $O(N^2 \cdot d)$ — the $N \times N$ attention score matrix is computed for every forward pass
- Memory complexity: $O(N^2)$ — the $N \times N$ attention matrix stored in HBM (THIS is the bottleneck)

★ EXAM: For $N=32K$ tokens: $N^2 = 1$ billion elements = 4 GB for FP32. For $N=128K$: 64 GB — larger than GPU HBM!

9.2 Why Standard Attention is Memory-Bound

Standard attention has 5 separate operations, each reading and writing from slow HBM:

Operation	HBM Read	HBM Write	Can be fused?
QK^T matmul	$Q (N \times d), K (N \times d)$	$S = QK^T (N \times N)$	No (output too large)
Masking	$S (N \times N)$	$S' = \text{Mask}(S) (N \times N)$	Yes, into matmul
Softmax	$S' (N \times N)$	$P = \text{softmax}(S') (N \times N)$	Yes, but needs all scores first
Dropout	$P (N \times N)$	$P'' (N \times N)$	Yes
PV matmul	$P'' (N \times N), V (N \times d)$	$O (N \times d)$	Yes

```
GPT-2 medium benchmark (seq_len=1024, head_dim=64, batch=64, A100):
Standard: HBM R/W = 40.3 GB, Runtime = 41.7 ms
FlashAttention: HBM R/W = 4.4 GB, Runtime = 7.3 ms
Reduction: 9.2× less HBM traffic, 5.7× faster despite 13% MORE FLOPs
Conclusion: Attention IS memory-bound (HBM traffic dominates, not FLOPs)
```

9.3 FlashAttention — The Two Key Innovations

Innovation 1: Kernel Fusion

Instead of 5 separate HBM read-compute-write operations, fuse everything into ONE kernel:

- Load Q, K, V tiles from HBM to SRAM ONCE
- Execute QK^T , masking, softmax, dropout, PV — all inside SRAM
- Write output O to HBM ONCE
- Result: HBM traffic $\propto O(N \cdot d)$ instead of $O(N^2)$

Innovation 2: Online Softmax (Tiled Softmax)

Standard softmax requires ALL N scores simultaneously (to compute denominator). Online softmax computes the exact same result incrementally, one block at a time, using just two running statistics per row:

- $m(x) = \max_i(x_i)$ — running maximum (for numerical stability)
- $\blacksquare(x) = \sum_i \exp(x_i - m(x))$ — running sum of shifted exponentials
- $\text{softmax}(x) = \exp(x_i - m(x)) / \blacksquare(x)$

Combining two blocks: When we see a new block with max $m_{\blacksquare}$ and sum $\blacksquare_{\blacksquare}$:

$$\begin{aligned} m_{\blacksquare_{\text{new}}} &= \max(m_{\text{old}}, m_{\blacksquare}) \\ \blacksquare_{\text{new}} &= e^{(m_{\text{old}} - m_{\blacksquare_{\text{new}}})} \cdot \blacksquare_{\text{old}} + e^{(m_{\blacksquare} - m_{\blacksquare_{\text{new}}})} \cdot \blacksquare_{\blacksquare} \\ O_{\text{new}} &= \text{diag}(\blacksquare_{\text{new}})^{-1} [\text{diag}(\blacksquare_{\text{old}}) \cdot e^{(m_{\text{old}} - m_{\blacksquare_{\text{new}}})} \cdot O_{\text{old}} + e^{(m_{\blacksquare} - m_{\blacksquare_{\text{new}}})} \cdot P_{\blacksquare} \cdot V] \end{aligned}$$

The $e^{(m_{\text{old}} - m_{\blacksquare_{\text{new}}})}$ and $e^{(m_{\blacksquare} - m_{\blacksquare_{\text{new}}})}$ terms **rescale** previous partial sums to account for the updated global maximum. This is mathematically exact — no approximation.

9.4 FlashAttention IO Analysis — Worked Tiling Example

This example demonstrates WHY tiling reduces HBM I/O even with the same FLOPs:

Setup

Q has 9 rows total. $|Q|$ = size of Q matrix = 18 units. $|K| = 18$, $|V| = 9$, $|O| = 9$

Standard Attention (no tiling): row-by-row processing

For each row of Q (9 rows total), must read ALL of K and V:

$$\begin{aligned} \text{IO} &= |Q| + r(Q) \times (|K| + |V|) + |O| \\ &= 18 + 9 \times (18 + 9) + 9 \\ &= 18 + 9 \times 27 + 9 \\ &= 18 + 243 + 9 = 270 \text{ IO units} \end{aligned}$$

FlashAttention with block size $b=3$ (reads 3 Q rows at once)

Number of Q blocks = $r(Q)/b = 9/3 = 3$ blocks (instead of 9 row reads)

$$\begin{aligned} \text{IO} &= |Q| + b(Q) \times (|K| + |V|) + |O| \\ &= 18 + 3 \times (18 + 9) + 9 \\ &= 18 + 3 \times 27 + 9 \\ &= 18 + 81 + 9 = 108 \text{ IO units} \end{aligned}$$

IO Reduction = $270 / 108 = 2.5\times$ with block size 3

Note: Same number of FLOPs! IO is reduced because K and V are loaded once per Q-block instead of once per Q-row.

A100 uses block size 8×8 (not 3). With $b=9$ (all rows at once):

$$\text{IO} = 18 + 1 \times (18+9) + 9 = 18+27+9 = 54 \text{ IO (5\times reduction)}$$

(Not achievable since $K+V+Q+O$ must fit in 20 MB SRAM)

✓ Larger blocks = more reuse of K and V = less HBM traffic. Block size limited by SRAM capacity.

9.5 Block Size Calculation — How Large Can Tiles Be?

The tiling block sizes are determined by SRAM capacity. At any time, one Q-tile, one K-tile, one V-tile, and one O-tile must fit in SRAM:

$$B_c = \lceil M / 4d \rceil \text{ [column block size = K/V tile size]}$$

$$B_r = \min(\lceil M / 4d \rceil, d) \text{ [row block size = Q/O tile size]}$$

Why $4d$? We need 4 tiles of size $B \times d$ in SRAM: Q_tile, K_tile, V_tile, O_tile.

Concrete calculation for A100 GPU:

$M = 20 \text{ MB SRAM} = 20 \times 10^6 \text{ bytes}$

$d = 64$ (head dimension in GPT-2 / standard transformer)

Each element = 2 bytes (FP16)

$$B_c = \lceil 20 \times 10^6 / (4 \times 64 \times 2) \rceil = \lceil 20 \times 10^6 / 512 \rceil = \lceil 39,062.5 \rceil = 39,063$$

$$B_r = \min(39,063, 64) = 64$$

Interpretation:

Q_tile: $64 \times 64 = 4,096 \text{ elements} \times 2B = 8 \text{ KB}$

K_tile: $39,063 \times 64 \times 2B \approx 5 \text{ MB}$ [ideally]

In practice, A100 uses 8×8 blocks due to alignment and warp constraints

9.6 FlashAttention Performance Results

Metric	Standard Attention	FlashAttention	Improvement
HBM R/W (GPT-2 med.)	40.3 GB	4.4 GB	9.2× less HBM traffic
Runtime (GPT-2 med.)	41.7 ms	7.3 ms	5.7× faster
GFLOPs (GPT-2 med.)	66.6	75.2	13% MORE (recomputation)
Memory complexity	$O(N^2)$	$O(N)$	10–20× less memory
Wall-clock speedup	—	2–4× (up to 6×)	Higher with masking+dropout
BERT training	20.0 min (MLPerf)	17.4 min	~15% faster
OOM threshold	$N \approx 2K$ on A100	$N = 128K+$ easily	Enables long-context models

★ EXAM: FlashAttention does NOT reduce FLOPs — it reduces HBM I/O. Same $O(N^2)$ math, different memory access pattern.

★ EXAM: Memory is LINEAR in N ($O(N)$) vs standard $O(N^2)$. This is what enables 128K+ context lengths.

9.7 Recomputation in the Backward Pass

Standard backprop through attention requires the $N \times N$ attention matrix P from the forward pass. FlashAttention does NOT store P (it's $N \times N =$ too large). Instead:

- **During forward pass:** Save only the softmax statistics (m and τ , each of size N) — $O(N)$ memory
- **During backward pass:** Recompute attention from Q, K, V + saved (m, τ) — same tiling algorithm
- **Cost:** ~13% more FLOPs (recomputing attention twice) — WELL worth the 10-20× memory savings

✓ Store $O(N)$ softmax stats; recompute $O(N^2)$ attention matrix on demand. Time-memory tradeoff in favor of memory.

MASTER FORMULA REFERENCE

All Critical Formulas — One Page

Formula	Name	Variables
$S = 1 / [(1-p) + p/s]$	Amdahl's Law	S=speedup, p=parallel fraction, s=improvement speedup
$E = T_{\blacksquare} / (P \times T_P)$	Strong Efficiency Scaling	$T_{\blacksquare}$ =single GPU time, P=#GPUs, T_P =parallel time
Attainable FLOPS = min(Peak, AI × BW)	Roofline Model	AI=arithmetic intensity [FLOP/byte], BW=memory BW
$AI = \#FLOP / DRAM_bytes$	Arithmetic Intensity	DRAM bytes = bytes actually fetched from off-chip DRAM
Crossover AI = Peak FLOPS / Memory BW	Roofline Crossover	Above this: compute-bound. Below: memory-bound
$V_t = \beta_{\blacksquare} V_{t-1} + (1-\beta_{\blacksquare}) g_t$	Adam 1st Moment	V = momentum, $\beta_{\blacksquare} \approx 0.9$, g = gradient
$S_t = \beta_{\blacksquare} S_{t-1} + (1-\beta_{\blacksquare}) g_t^2$	Adam 2nd Moment	S = variance, $\beta_{\blacksquare} \approx 0.999$
$V_{\blacksquare}_t = V_t / (1-\beta_{\blacksquare}^t)$, $S_{\blacksquare}_t = S_t / (1-\beta_{\blacksquare}^t)$	Adam Bias Correction	Critical at early steps (t is small)
$w_{t+1} = w_t - \alpha \cdot V_{\blacksquare}_t / (\sqrt{S_{\blacksquare}_t} + \epsilon)$	Adam Update	α =learning rate, $\epsilon=1e-8$ (numerical stability)
$i = blockIdx.x * blockDim.x + threadIdx.x$	CUDA 1D Global Index	The most important CUDA formula
Row= $blockIdx.y * blockDim.y + threadIdx.y$, Col= $blockIdx.x * blockDim.x + threadIdx.x$	CUDA 2D Index	For 2D grids
Occupancy = Active Warps/SM / Max Warps/SM	SM Occupancy	Target: 100%; Warp = 32 threads
Attention(Q,K,V) = softmax(QK ^T / $\sqrt{d_k}$)V	Self-Attention	d_k =head dimension; divide to prevent saturation
$Bc = \lceil M/4d_{\blacksquare} \rceil$, $Br = \min(\lceil M/4d_{\blacksquare} \rceil, d)$	FlashAttention Block Sizes	M=SRAM size, d =head dimension
$IO = Q + b(Q)(K + V) + O $	FlashAttention IO Cost	$b(Q)$ =#Q-blocks, smaller→more HBM traffic
Ring AllReduce: $2N(P-1)/P$ per GPU	AllReduce BW	N=params, P=#GPUs; $\approx 2N$ for large P
PS comm at server: $2N(P-1)$	Parameter Server BW	Grows linearly with P — bottleneck!
ZeRO-3 mem/GPU: $(2+2+12)\psi/N$	FSDP Memory	ψ =params, N=#GPUs; 64× reduction with N=64
Bubble = $(P-1)/M$	Pipeline Bubble Fraction	P=pipeline stages, M=microbatches
$S = T_A/T_B$	Speedup	T_A =baseline time, T_B =optimized time
$E_{\text{scaling}} = t_{\text{serial}}/(t_{\text{parallel}} \times p)$	Scaling Efficiency	p=#processors; ideal=1.0
$x_G = (\prod x_{\blacksquare})^{(1/n)}$	Geometric Mean	Use for speedup ratios and benchmark comparisons

Critical Numbers to Memorize

Item	Value	Context
Warp size	32 threads	Always — scheduling unit in all CUDA GPUs
Max threads/block	1024	Cannot exceed this in kernel launch config
Shared mem banks	32, 4-byte granule	Bank conflicts kill shared memory performance
FP32→FP16 reduction	2×	Half the memory, same value range for BF16
FP32→INT8 reduction	4×	4× memory savings for inference
FP8 E4M3 max value	448	For forward pass (weights/activations)
FP8 E5M2 max value	57,344	For backward pass (gradients — needs more range)
Adam β_1 default	0.9 (LLaMA/GPT)	1st moment smoothing
Adam β_2 default	0.95–0.999	2nd moment smoothing
DAXPY arithmetic intensity	0.083 FLOP/byte	Memory-bound: 2 FLOPs, 24 bytes
7-point stencil AI	0.109 FLOP/byte	Still memory-bound
FlashAttention speedup	2–4× (up to 6×)	GPT-2 medium: 5.7×
FlashAttention HBM reduction	9.2× (GPT-2 med.)	40.3 GB → 4.4 GB
Standard attention memory	$O(N^2)$	N=sequence length
FlashAttention memory	$O(N)$	Linear! Enables 128K+ contexts
ZeRO-3 reduction (N=64)	64×	120 GB → 1.9 GB per GPU for 7.5B model
Ring AllReduce cost/GPU	$2N(P-1)/P \approx 2N$	Independent of P for large P
A100 HBM bandwidth	2 TB/s (HBM2e)	Critical for memory-bound kernels
H100 HBM bandwidth	3.35 TB/s (HBM3)	1.7× more than A100
SRAM bandwidth	~19 TB/s	~13× faster than HBM
Adam memory per param	16 bytes	$2(\text{wt})+2(\text{grad})+4(\text{master})+4(\text{mom})+4(\text{var})$